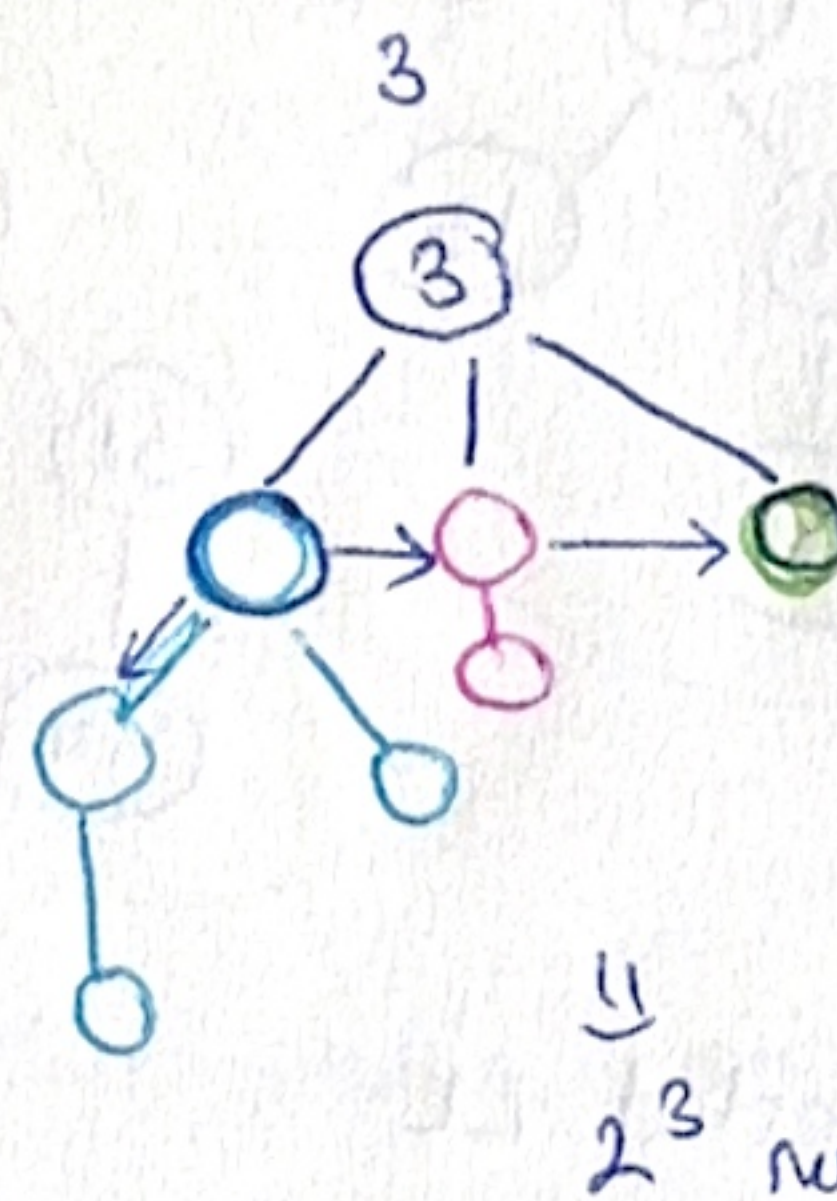
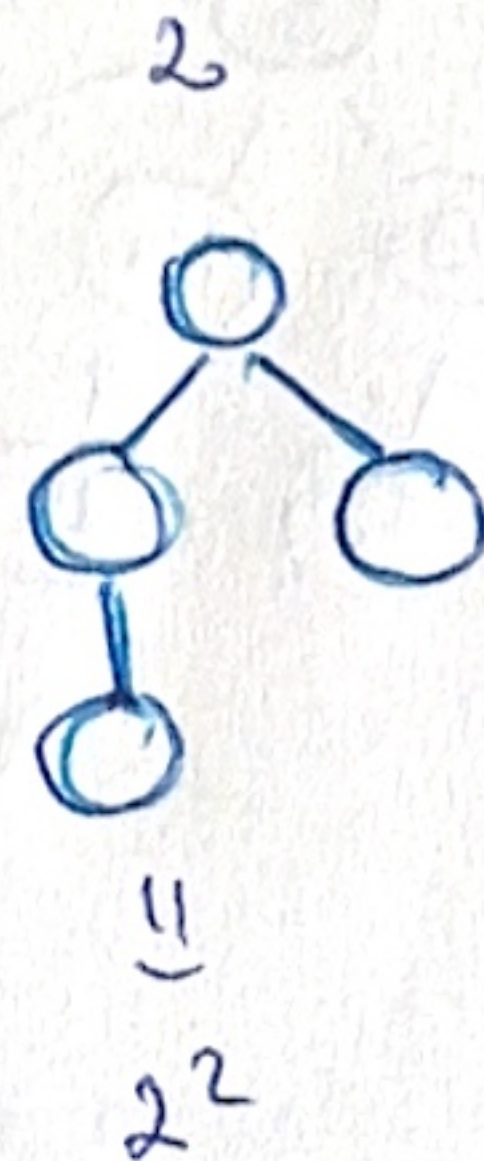


1) Binomial heap = collection of binomial trees.

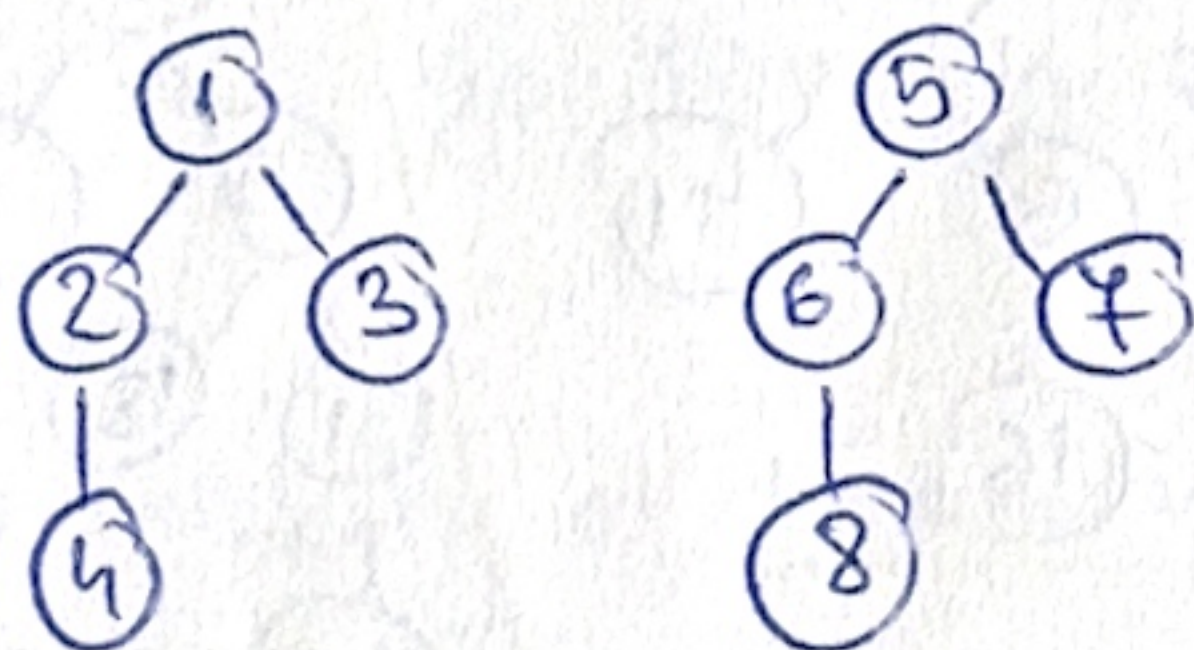
BT: order 0
binomial tree
binary tree? 

Nr. nodes = pow of 2

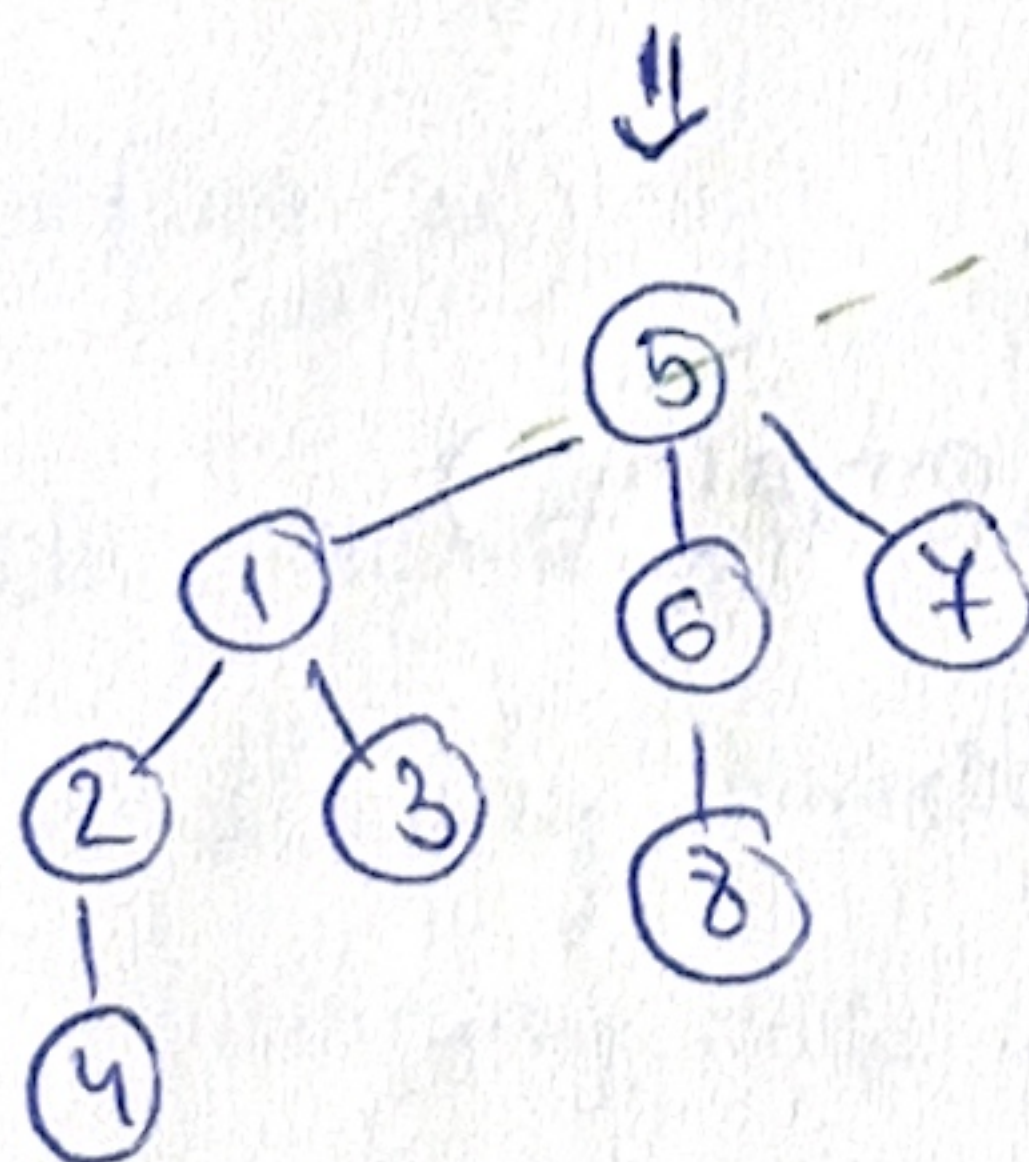


4 nodes: 4 + 2 + 1

2 bt of order 2. how to merge them? in $\Theta(1)$ time.



$\Rightarrow$ they respect the min-heap property.
 $\Rightarrow$ make 5 the leftmost child of 1
 $\Rightarrow$ the property is respected.



Remove the root $\Rightarrow$ valid BT.

2) BT implementation

- keep adr. of root node
- each node holds adr. of the one on its right below

How many BT inside a BH?

$\rightarrow$ Look at powers.

4 nodes = $2^2 + 2^1 + 2^0 \Rightarrow 3 \text{ BT.}$

122 nodes = $1 + 2 + 4 + 8 + 16 + 25 + \dots$

(SUM OF POWERS OF 2) $\Rightarrow \dots$

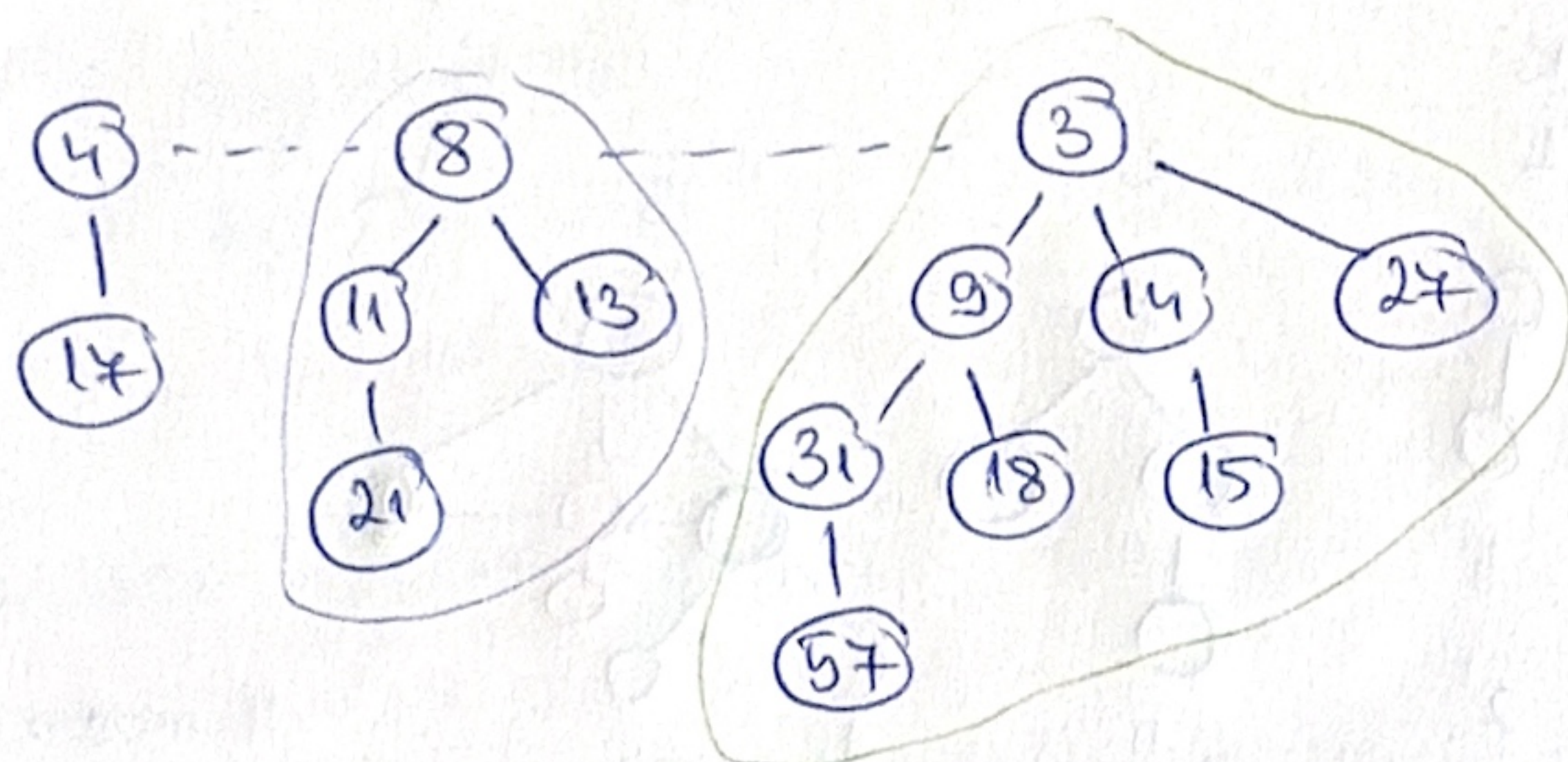
Quiz 9: 37 nodes = $2^5 + 2^2 + 2^0$
 $= 32 + 4 + 1$
 $= 37.$

3 trees.

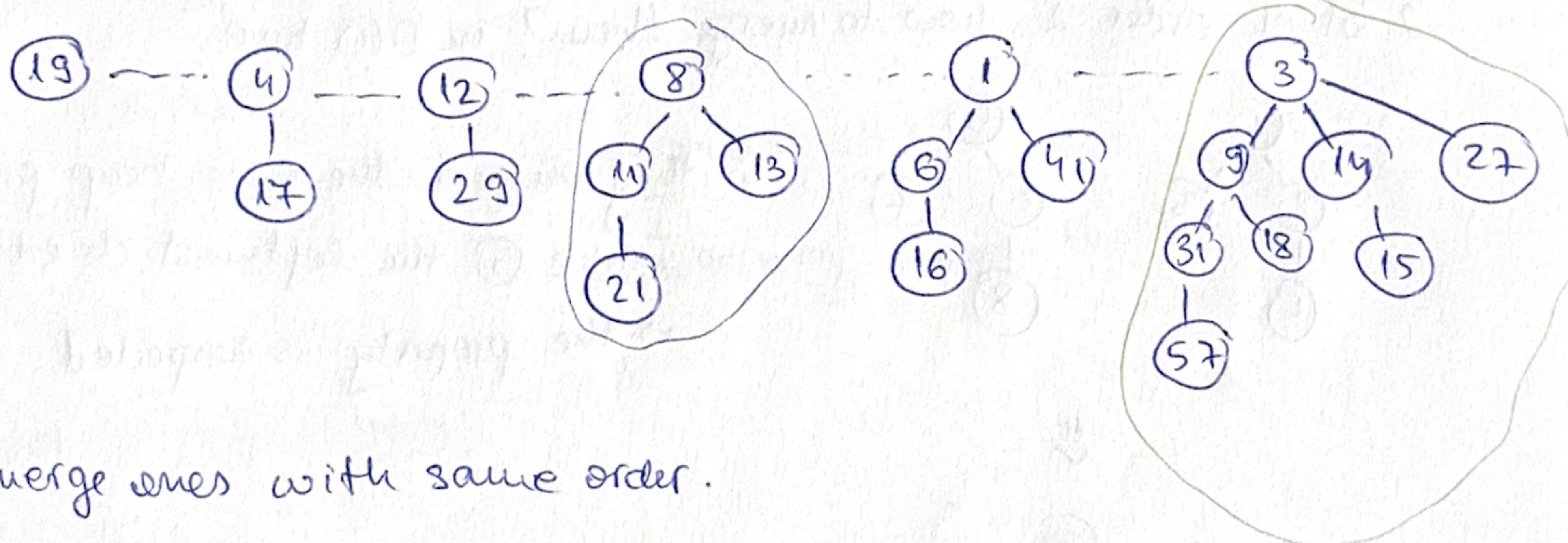
$$\begin{array}{r} 32 + \\ 16 \\ \hline 48 \end{array}$$

3) Operations

MERGE - two trees ✓
- two heaps

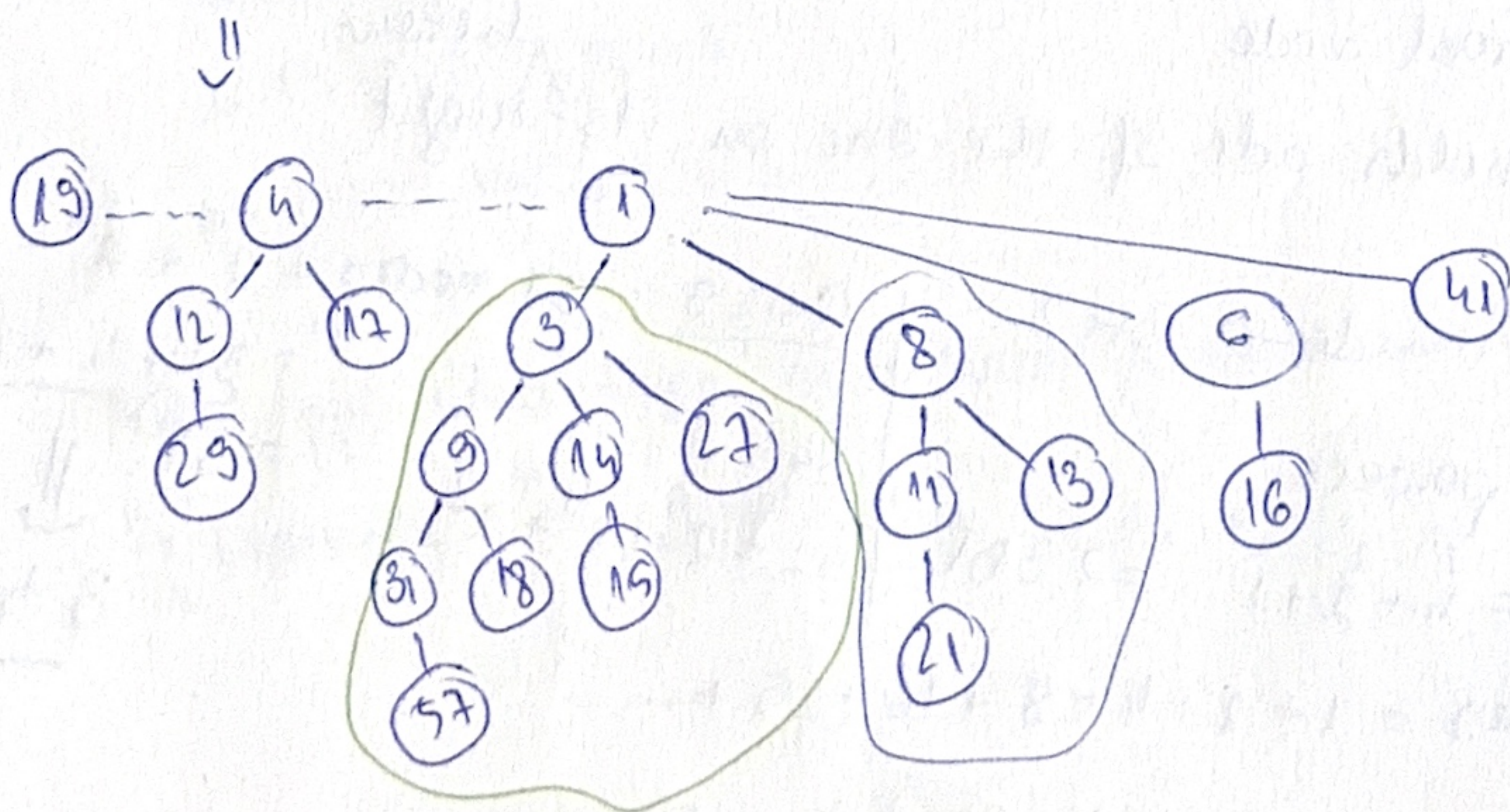
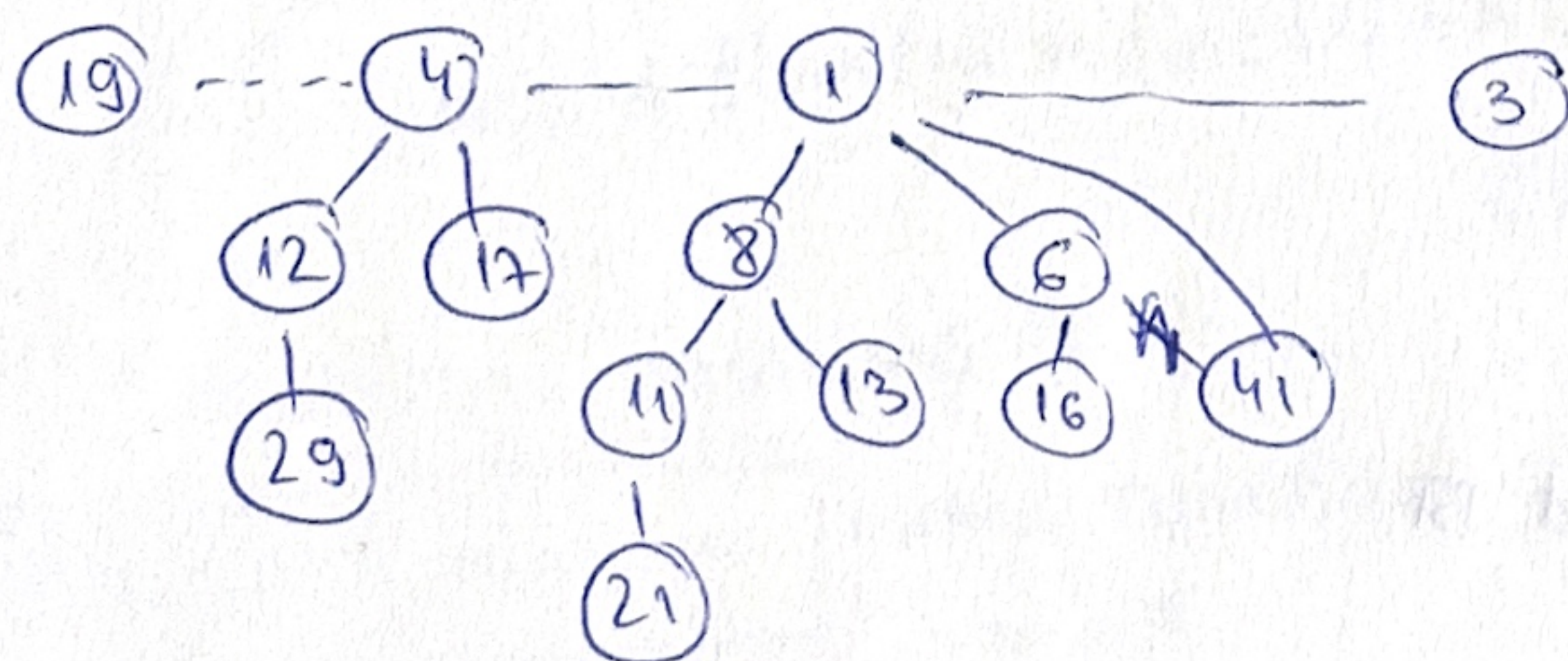


Merge sorted LL



merge ones with same order.

↓ (keeping in mind the property)



• Complexity of merging: $O(\text{nr of trees } (H_1) + \text{nr of trees } (H_2))$

$\Rightarrow$ logarithmic $\Rightarrow O(\log_2 m)$?

$\Rightarrow$ better complexity than ...

PUSH

POP

TOP

$\rightarrow$ Complexity of adding: "pretty much the same" as merge ...?

$\Rightarrow$ not easier than merging

$\Rightarrow$ wc: go through everything

$\rightarrow$ Complexity of removing:

$\Rightarrow$ ex: remove the root, $\leftrightarrow$ swap $(3) \leftrightarrow (41) \Rightarrow$ valid heap.

$\Rightarrow O(\log_2 m)$

4) Public transportation service

$\rightarrow$ display timetable

$\rightarrow$ display info for bus line

$\rightarrow$ fast! $\Rightarrow$ array $\Rightarrow$ keep them sorted

$\rightarrow$ rename bus lines

use bus nr as index!

$\Rightarrow$ DAT_{able}

simple / efficient

$\Theta(1)$

ADD new line

$\Rightarrow$ index 72 in DAT

HASH TABLE

based on arrays

two $\neq$ keys produce the same array index

issues:

$\rightarrow$ 24B

$\rightarrow$ memory

might not be numbers collisions (normal)

$\rightarrow$ function that gives position $0, m-1 \Rightarrow h(\text{key}) \rightarrow 0 \dots m-1$

$T[\text{key}] \leftarrow \text{value (DAT)}$

$T[h(\text{key})] \leftarrow \text{value (HT)}$

(3)

(7)

5) What makes a good hash function?

→ minimize collisions

→ deterministic

→ $\Theta(1)$ time

→ assumption of simple uniform hashing

• DIVISION METHOD: $h(k) = k \bmod m$

$m \rightarrow$ parameter $\rightarrow$ hash table size

capacity $\rightarrow$ primes m , not too close to 2^p or 2 .

← in case we want big values (CNP) represented in the array?

• MID-SQUARE METHOD:

$m \rightarrow \dots$
 $h(4567)$

$$4567 * 4567 = 20857489 = 57$$

↓
most mixing happens here!

• MULTIPLICATION METHOD:

$m \rightarrow \dots$

fractional, floor:

fractional part

$$h(k) = \text{floor}(m * \text{frac}(k * A))$$

↓
 $\in (0,1)$

UNIVERSAL HASHING?

$$h(k) = k \bmod m$$

HASH FUNCTION

most used

$$m = 7$$

$$h(10) = 10 \% 7 = 3$$

$$h(3) = 3 \% 7 = 3$$

$$h(17) = 17 \% 7 = 3$$

} collisions

Avoid ppl who take your hash function to cause collisions

↳ Example 1st, Example 2nd, 3

6) what if keys are not numbers? How could we define our hash function?

→ hash functions specific for numbers (a lot)

I. Key ^{convert} → number

II.

III. multiply every numerical code with

$$\underbrace{s[0]} \times 26^{m-1} + \underbrace{s[1]}$$

from the string

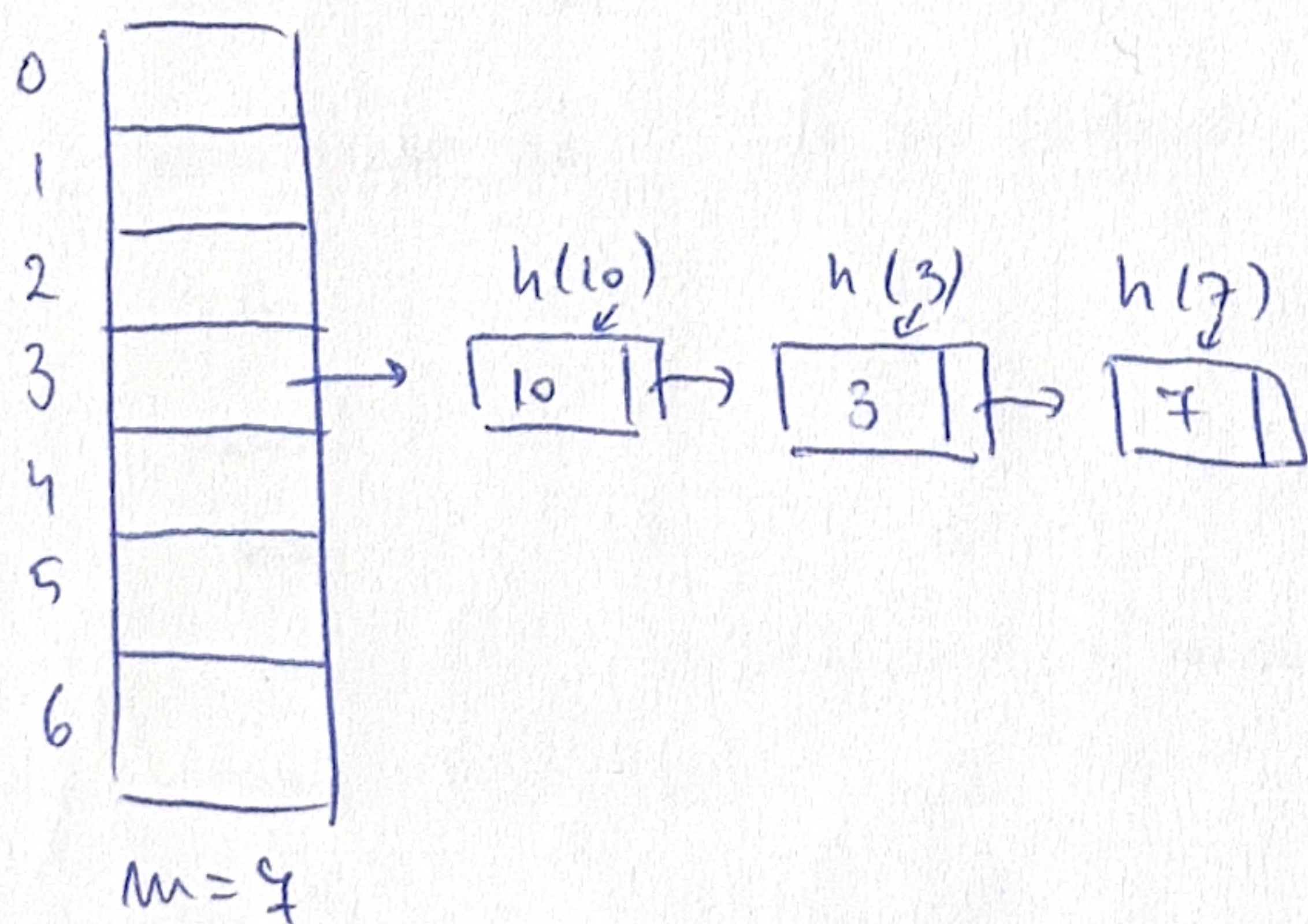
7) new password? Cryptography

↳ they don't know it → they pass it to the hash function →
→ see if they match

$$\left. \begin{matrix} h(x) = h(y) \\ x \neq y \end{matrix} \right\} \Rightarrow \text{collision.}$$

8) birthdays

- 367 ppl ⇒ at least 2 w same bd.
- 23 ppl are enough for 50%.



separate chaining
($\alpha > 1$)

we don't want

$$O(0.5) \downarrow \alpha$$

Complexity of searching: $O(\text{length of LL}) \sim O(\alpha + 1)$

$\alpha = \frac{m}{n}$ how "full" the hash table is.

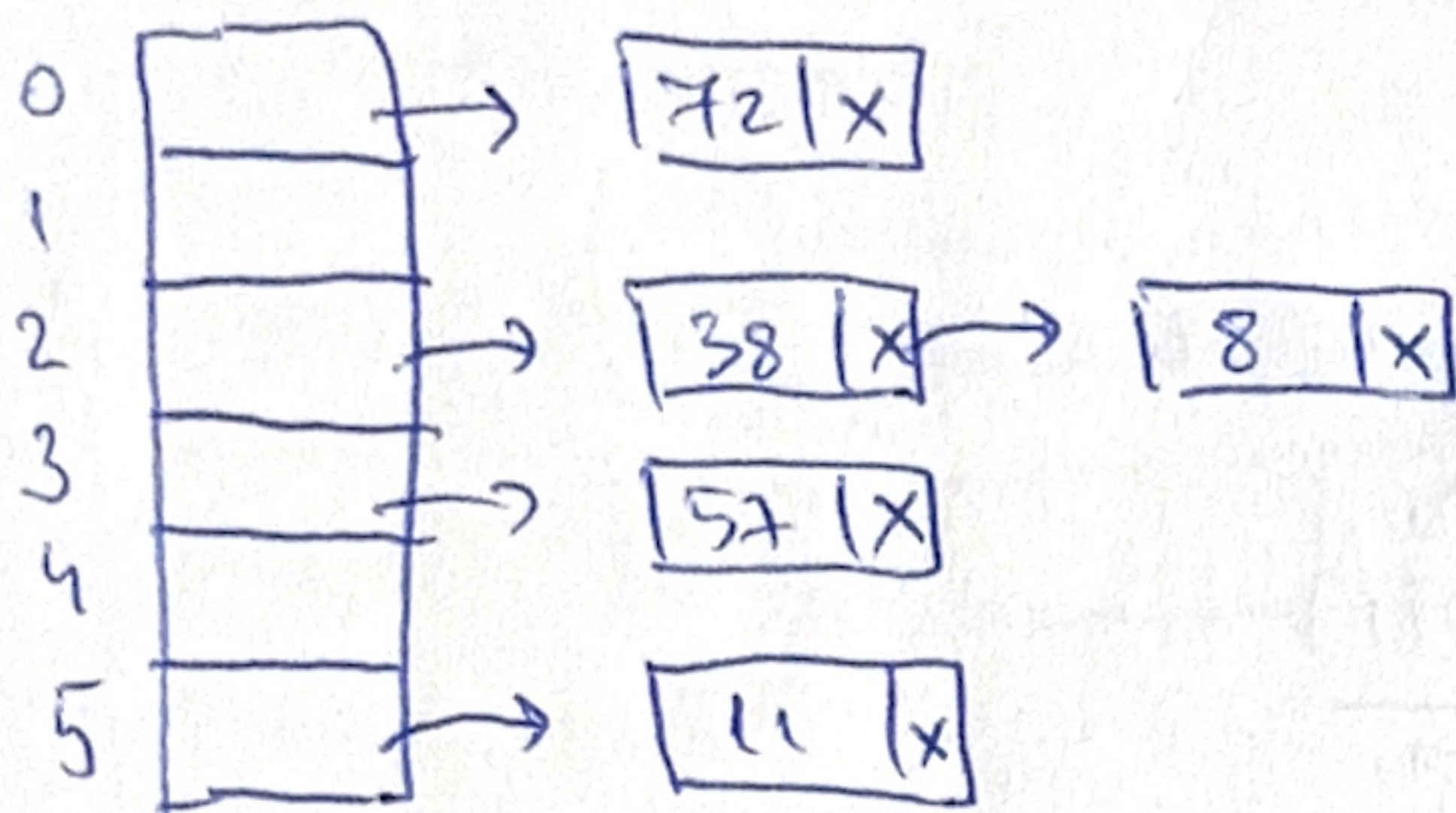
AVERAGE
COMPLEXITY

$\frac{3}{7}$ ↳ AVG length of a list

! Complexities on average are constant as long as α is small enough.

Exam: Collision-resolution method:

$m = 6$



→ insert elements in hash table

$$h(38) = 38 \% 6 = 2, \quad \alpha = \frac{1}{6} < 0.7.$$

$$h(11) = 11 \% 6 = 5 \rightarrow \text{empty pos}, \quad \alpha = \frac{2}{6} < 0.7$$

$$h(8) = 8 \% 6 = 2, \quad \alpha = \frac{3}{6} < 0.7.$$

$$h(72) = 72 \% 6 = 0, \quad \alpha = \frac{4}{6} < 0.7.$$

$$h(57) = 57 \% 6 = 3, \quad \alpha = \frac{5}{6} > 0.7 \quad \underline{\text{STOP}}$$

→ resize, rehash

$$h(11) = 11 \% 12 = 11 \quad (\text{search for 11}) \Rightarrow \text{not there although it is there (problem!)}$$

↓
new
m